

Lecture 01

2026-03-31

Introduction

Hello & Welcome to Stats 20

- Please greet your neighbor(s)
- My name is Dr. Vivian Lew, I am a senior lecturer in the Department of Statistics & Data Science
- Statistics is about learning from data - drawing conclusions, assessing uncertainty, generalizing results to a larger population
- Data Science is about using data to make predictions/automate decisions
- R is the language of Statisticians
- The goal is to prep you for upper division work in Statistics & Data Science
- First Day Stuff today

What to Expect this quarter

Format

- This class is project-based, hands-on approach to developing computational problem solving approaches, concepts, and skills in R
- I ask you to try things while in class and to ask questions
- So please bring a laptop or comparable and be prepared to use it
- Grades are assigned on the university's straight scale (90-100% A type of grade, 80-89% B type of grade, etc.)

Deliverables

- 5% Group “Meet the Professor” (in person, before the end of Week 3)
- 10% DataCamp (multiple assignments per week, graded complete/not complete)
- 10% Weekly BruinOnline Quiz (graded complete/not complete)
- 10% Basic Reproducible Report (Due Week 3)
- 10% Video Recorded Presentation (Due Week 5)
- 10% Polished Reproducible Report (Due Week 7)
- 15% Discussion Section and Section Project (Due Week 9 or later, TAs decision)
- 20% Final Exam (Friday, Week 11, more to follow)
- 10% Attend 1 class meeting per week (1-10) **in person**

Choose your tools

- I will be using RStudio. It is an Integrated Development Environment (IDE) for R
- You are free to use whatever IDE you want (RStudio *preferred*, Positron, VS Code, JetBrains) to complete assignments
- I will ask your TAs to assist you with RStudio but do decide on your tools before the end of Week 2 (first assignment that needs it is due Week 3)
- You are free to obtain assistance from your favorite LLM/AI (READ THE COURSE AI POLICY FIRST)
- The point of an IDE like RStudio or whatever is to teach familiarity with software that allows you to write scripts (.R) and notebooks (preferably .qmd but .rmd will work), manage them, and execute/render/run them.
- Note: your tool choice may affect how your results are delivered (e.g., PDF, web page)

AI Policy

- We permit and encourage the **responsible** use of AI-assisted programming tools (e.g., GitHub Copilot, built-in IDE assistants or similar tools which have been integrated into RStudio, Positron or others).
- Presently, programming in professional (paid/internship) environments usually involves some AI-support.
- Learning how to use these tools well (e.g., understand the coding decision, have an eye to correcting it or changing suggestions) and ethically is part of become a competent programmer in any language since 2023.

- Please review the syllabus and/or the recording of it for an expanded detailed version of this summary

Flying Over Stats 20

Outline

Over the next 10 weeks, we will try to cover the following topics:

1. Data Science Workflow (computational steps), Reproducibility, Communication.
2. Core/Base R (the language, data structures, functions, etc.).
3. Data Wrangling (not analyzable -> analyzable).
4. Debugging/Problem Solving.
5. Tidyverse + Data Viz
6. Distributions, Sampling, Statistics.
7. Control Flow, Functions, and Programming Concepts.
8. Linear Algebra in R.

Workflow (in research)

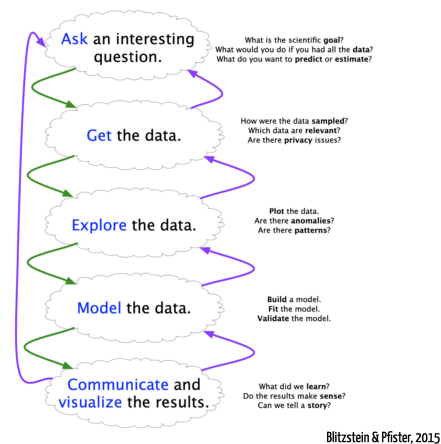


Figure 1: A Data Science Workflow

Workflow (in workplaces)

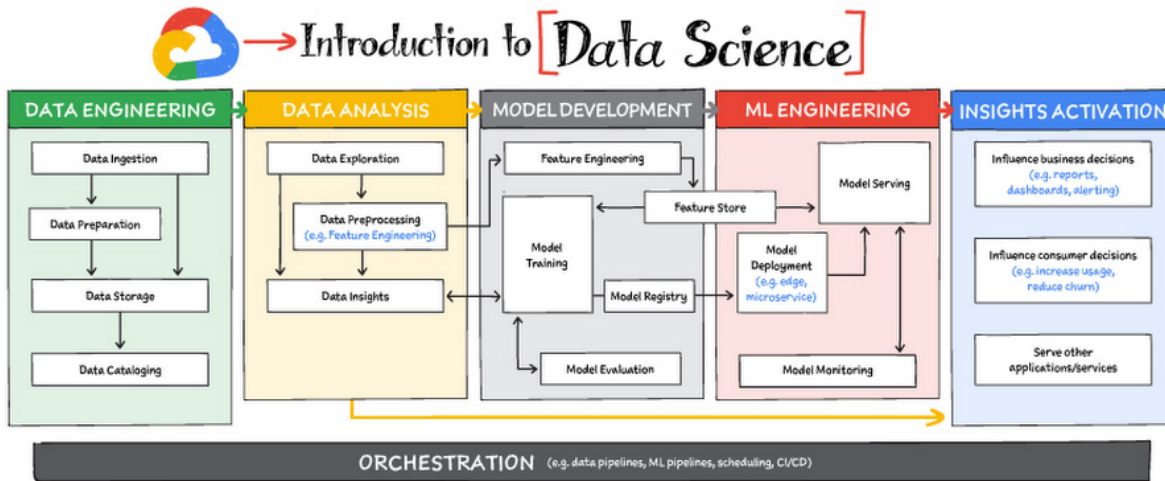


Figure 2: From Google

What is R?

From CRAN (the Comprehensive R Archive Network):

- R is a system for statistical computation and graphics. It consists of a language plus a run-time environment with graphics, a debugger, access to certain system functions, and the ability to run programs stored in script files.

Translation:

- R is a language with an interpreter and a standard library (toolkit/toolbox of functions)
- With execution support code (behind the scenes code that helps your R code run on your computer)
- All for the practice of statistics (involves computation, simulation, data visualization etc.)
- R can also monitor/analyze code, identify coding errors, interact with your operating system, run programs saved on your computer as scripts (or similar) among other things.

Activity #1 R in the Browser

- Sometimes, like today, I want all of you to have access to R at your fingertips like on your laptop or on your phone or tablet.
- Today, we will use [R in the Browser](#).

- This is a simple webpage version of R, not really for analysis, more for instruction by example

Activity #1 Takeaways

- In the beginning, R is like a calculator
- Numbers, operators, vectors, data frames, functions are objects.
- A simplification: virtually everything in R is an object.
- Names like x, y, z, iris, rivers are associated with R objects and by calling the name, we have access to the associated object
- The fundamental data structure in R is the vector
- Vectors are indexed (elements are numbered 1 to however many elements we have)

The “BIG” Question

Why R and not Python?

- The basic difference: R was built by statisticians for statisticians, so the focus is narrower.
- Python was built by software engineers for general purpose applications, so the focus is broad.
- R is great for focusing on thinking about statistical problems, solving them, and communicating the results
- Python is great for doing lots of different things. Example, download thousands of YouTube videos, analyze the images frame by frame, transcribe the sound into words, analyze the data, etc.

Why R and not Python? (cont’d)

- R is “Problem-First” or “Problem-Forward”
- In R, data structures are integrated directly into the language and do not require additional tools (but tools, found in libraries, are helpful).
- When you load a dataset, R assumes you want to analyze it. The syntax is designed to mirror the way a researcher thinks: “I want to study something which involves data, get the data, then filter/subset it, then group it, then summarize/describe, then model, etc.”
- Also R’s simpler modeling formulas allows one to remain thinking about the analytical problem.

Why R and not Python? (cont'd)

- Python is “System-First”
- In Python, you “build” your environment before you can solve the problem (importing tools, found in libraries such as pandas, numpy, etc.)
- In Python, it matters how your programming code integrates within a larger system, the result is versatility. So it is easy for Python to interact with YouTube for example (not as easy/or maybe not possible in R)
- In R, what matters is how the programming code **helps you reveal the truth found in the data.**

Visual



R v Python Example 1

Let's look at a bivariate (simple) regression: one Y (dependent), one X (independent). If you have forgotten see [IMS Chapter 7](#).

R

If you already have R running on your computer or copy and paste this to the activity, you can try this

```
model1 <- lm(mpg ~ wt, data = mtcars)

summary(model1)
```

Call:

```
lm(formula = mpg ~ wt, data = mtcars)
```

Residuals:

Min	1Q	Median	3Q	Max
-4.5432	-2.3647	-0.1252	1.4096	6.8727

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	37.2851	1.8776	19.858	< 2e-16 ***
wt	-5.3445	0.5591	-9.559	1.29e-10 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 3.046 on 30 degrees of freedom

Multiple R-squared: 0.7528, Adjusted R-squared: 0.7446

F-statistic: 91.38 on 1 and 30 DF, p-value: 1.294e-10

Python

```
import statsmodels.api as sm

mtcars = sm.datasets.get_rdataset("mtcars", "datasets").data

X = mtcars["wt"]
```

```
X = sm.add_constant(X)
Y = mtcars["mpg"]

fit = sm.OLS(Y, X).fit()

print(fit.summary())
```

```

=====
                        OLS Regression Results
=====
Dep. Variable:          mpg      R-squared:                0.753
Model:                  OLS      Adj. R-squared:            0.745
Method:                 Least Squares    F-statistic:          91.38
Date:                  Tue, 31 Mar 2026    Prob (F-statistic):    1.29e-
10
Time:                  17:01:31    Log-Likelihood:        -
80.015
No. Observations:      32      AIC:                  164.0
Df Residuals:          30      BIC:                  167.0
Df Model:              1
Covariance Type:       nonrobust
=====

```

	coef	std err	t	P> t	[0.025	0.975]
const	37.2851	1.878	19.858	0.000	33.450	41.120
wt	-5.3445	0.559	-9.559	0.000	-6.486	-

```

=====
Omnibus:                2.988    Durbin-Watson:          1.252
Prob(Omnibus):          0.225    Jarque-Bera (JB):        2.399
Skew:                   0.668    Prob(JB):                0.301
Kurtosis:               2.877    Cond. No.:               12.7
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

Takeaways

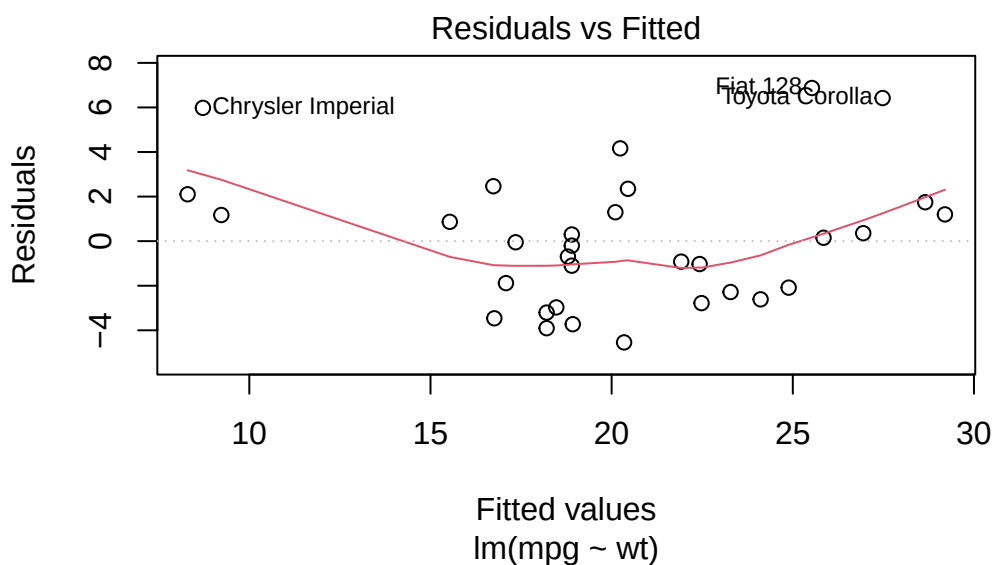
- R has “built-in” data “data = mtcars”, Python does not, need a library.
- In both situations, we are testing the same hypothesis: does MPG change in a predictable direction as WT changes?

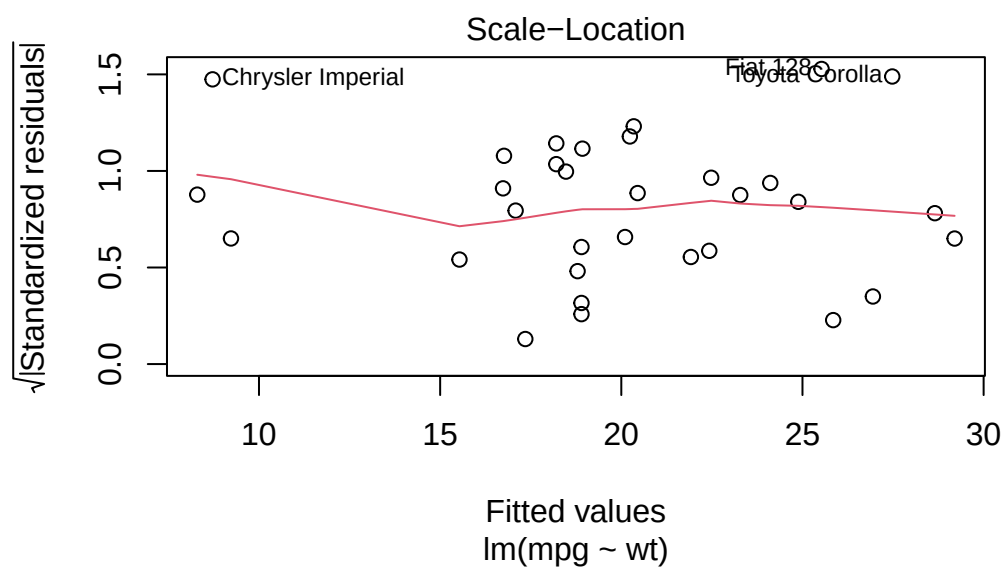
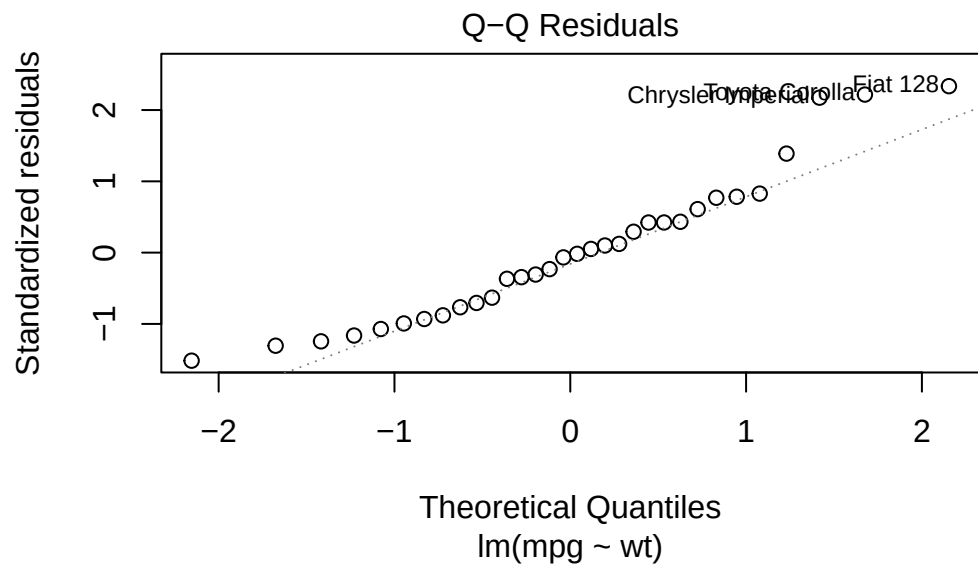
- Notice the “<-” in R this is called a “gets”, it is used for assignment. Could read it as “model 1 gets”
- Python uses “=” for both assignment and options
- You can use “=” for assignment in R, but it is really meant to identify options
- R is written in a way comparable to how we think about things in statistics $lm(mpg \sim wt, data = mtcars)$ **lm()** is the basic linear model **FUNCTION** (important word) and the () encloses its “arguments”. Functions do all of the “work” in R.
- ~ could be read “as a function of” so **we** can read the code like “linear model(mpg as a function of wt the data is mtcars)” and “model1” “gets” the result.
- You don’t need to print() explicitly in R, but you **must** explicitly print() in Python to see results

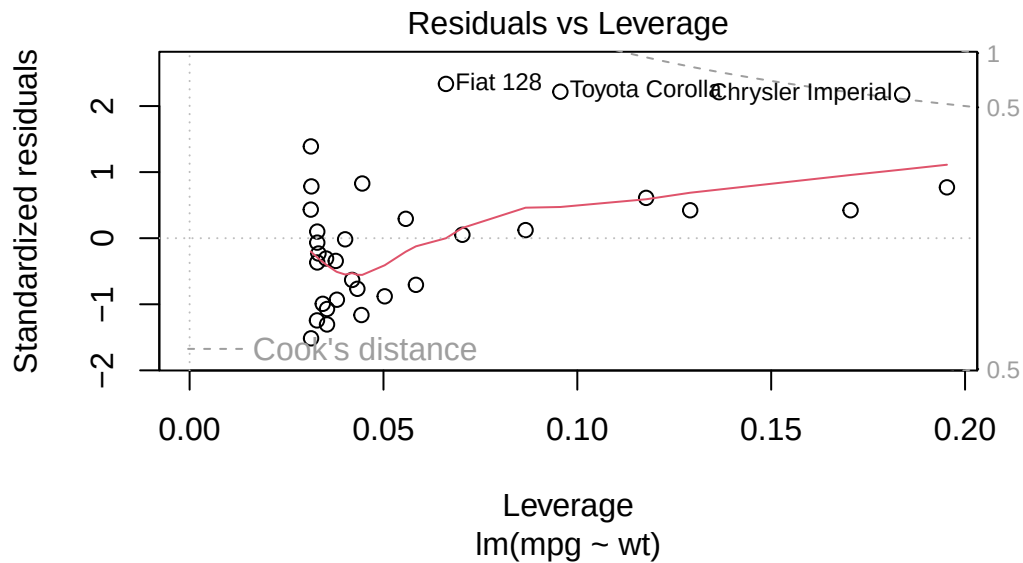
R

Diagnostic plots are part of the output of `lm()`

```
model1 <- lm(mpg ~ wt, data = mtcars)
plot(model1)
```







Python

You want diagnostic plots? This is for ONE plot in Python:

```
import numpy as np
import matplotlib.pyplot as plt
import statsmodels.api as sm

mtcars = sm.datasets.get_rdataset("mtcars").data
X = sm.add_constant(mtcars["wt"])
y = mtcars["mpg"]
model = sm.OLS(y, X).fit()

fitted = model.fittedvalues
residuals = model.resid

influence = model.get_influence()
standardized_resid = influence.resid_studentized_internal
leverage = influence.hat_matrix_diag
cooks = influence.cooks_distance[0]

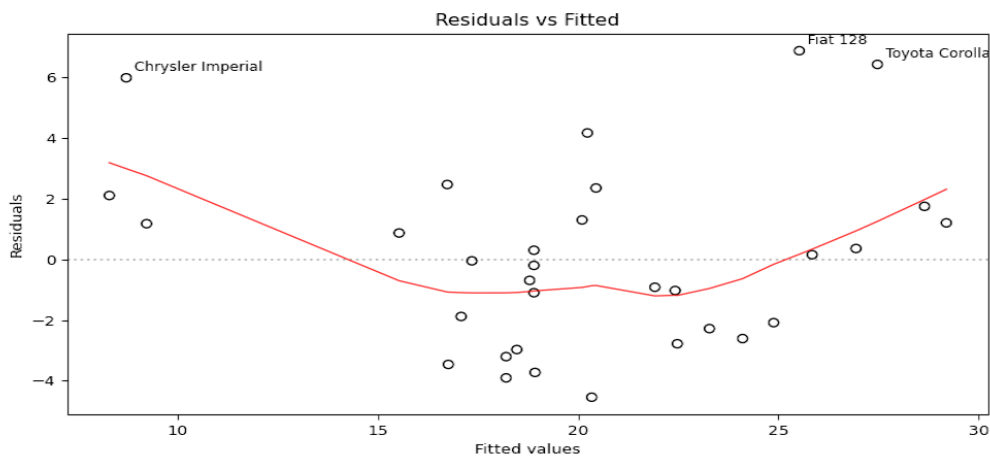
fig, ax = plt.subplots(figsize=(8, 5), dpi=100)

ax.scatter(model.fittedvalues, model.resid, alpha=0.7)
ax.axhline(0, color='red', linestyle='--')
```

```
ax.set_xlabel("Fitted values")
ax.set_ylabel("Residuals")
ax.set_title("Residuals vs Fitted")
plt.tight_layout()
```

Python Plot Result

- Python is very powerful and can make its plot look exactly like R's plot!
- This gives you a sense of the “swiss army knife” of Python



The “Formula” vs. “The Matrix”

In R, a statistical model is treated like a scientific hypothesis. In Python, a statistical model is treated like “how does the computer wants its input formatted?”

- **R Approach:** You use **formulas** ($y \sim x$). This is like saying “I think y changes in response to x .” R automatically handles things creating the underlying matrix and keeps the names of your variables intact so you can read the results.
- **Python Approach:** Most Python libraries(toolkits) require you to manually transform your data into a matrix of numbers first. Then pass them in as x and y .

In R, the code looks like the math inside your head. In Python, the code looks like the way the computer understands math.

R v Python Example 2

R allows for a more linear flow of thought, less engineering, more thinking. Not good or bad, just a different philosophy. In each of these examples, we filter (subset/split up), aggregate (combine), then analyze:

R

R uses the “pipe” operator (`%>%` Tidyverse library or `|>` core R) which is read like “then”. You know zero R today, but with a little help, you can read your R code like a sentence.

```
library(dplyr) # a library is like a toolbox filled with helpful functions
df %>% # df stands for "data frame", you'll learn more about this
  filter(year > 2020) %>% # filter() function - keep these years in the data frame
  group_by(state) %>% # Then, "group the data by" state for analysis
  summarize(mean_yield = mean(y)) # Finally, calculate the mean() save it to a new variable
```

We created a new variable just by naming it.

Python

In Python, you frequently create “intermediate” variables or use complex nested brackets to do the same:

```
from collections import defaultdict

## filter
filtered = [row for row in data if row["year"] > 2020]

## group and accumulate sums and counts
sums = defaultdict(float)
counts = defaultdict(int)

for row in filtered:
    state = row["state"]
    y = row["y"]
    sums[state] += y
    counts[state] += 1

## compute means
mean_yield_by_state = {
```

```

        state: sums[state] / counts[state]
    for state in sums
}

## print the result
for state, mean_yield in mean_yield_by_state.items():
    print(state, mean_yield)

```

Python with pandas

This is why pandas exists (compare Python using pandas against the previous Python no pandas)

```

import pandas as pd

## assume df is a data frame in Python
result = (
    df[df["year"] > 2020]          ## Keep these years
      .groupby("state", as_index=False) ## aggregate by state
      .agg(mean_yield=("y", "mean"))  ## Calculate the mean of y and store as mean_yield
)

print(result)

```

Python one line

Python pandas is still readable even when complex

```

import pandas as pd

result = df[df["year"] > 2020].groupby("state")["y"].mean().reset_index(name="mean_yield")

```

Questions?

If you don't want to raise your hand



- You are not required to participate
- Participation is always anonymous
- But sometimes I will ask the class questions to “take a pulse”
- Mentimeter is free to you, you just need a phone or a tablet or even a laptop

Where are we?

- Learn how to be a “System Architect” – someone who understands how code (regardless of the specific language) serves a specific business or research goal (this AI cannot do yet). Study/ understand the process in addition to trusting it.
- Focus on possessing AI Augmentation Skills – where AI is used as a tool to help humans do more, rather than replace humans. When coding in this class, AI is your helper to

develop your computational thinking and broaden your understanding of the application of the code beyond the problem in front of us.

- AI is the best at “book learning” (explicit knowledge) so instead of competing – leverage AI to help you gain **practical, hands-on experience** through projects and then internships and then jobs. Show that you can **apply** what you have learned in the areas/industries/fields that you want to join.
- Develop your “Tacit Knowledge” – an advanced form of pattern recognition that the brain does subconsciously after years of experience - having good judgment arising from “the feel” of a situation, for example - handling complex client interactions, dealing with “messiness” (when things do not go as planned). Develop your Spidey Senses for a situation.

Computational Thinking

The Value of Computational Thinking

- Decomposition - Breaking a larger problem down into smaller pieces
- Pattern Recognition - Looking for similarities within and across problems
- Abstraction - Avoid hyperfocusing on granular details and focusing instead on developing general solutions
- Algorithm Design - Automating the process efficiently, ensuring reproducibility and reducing the need for excessive human intervention

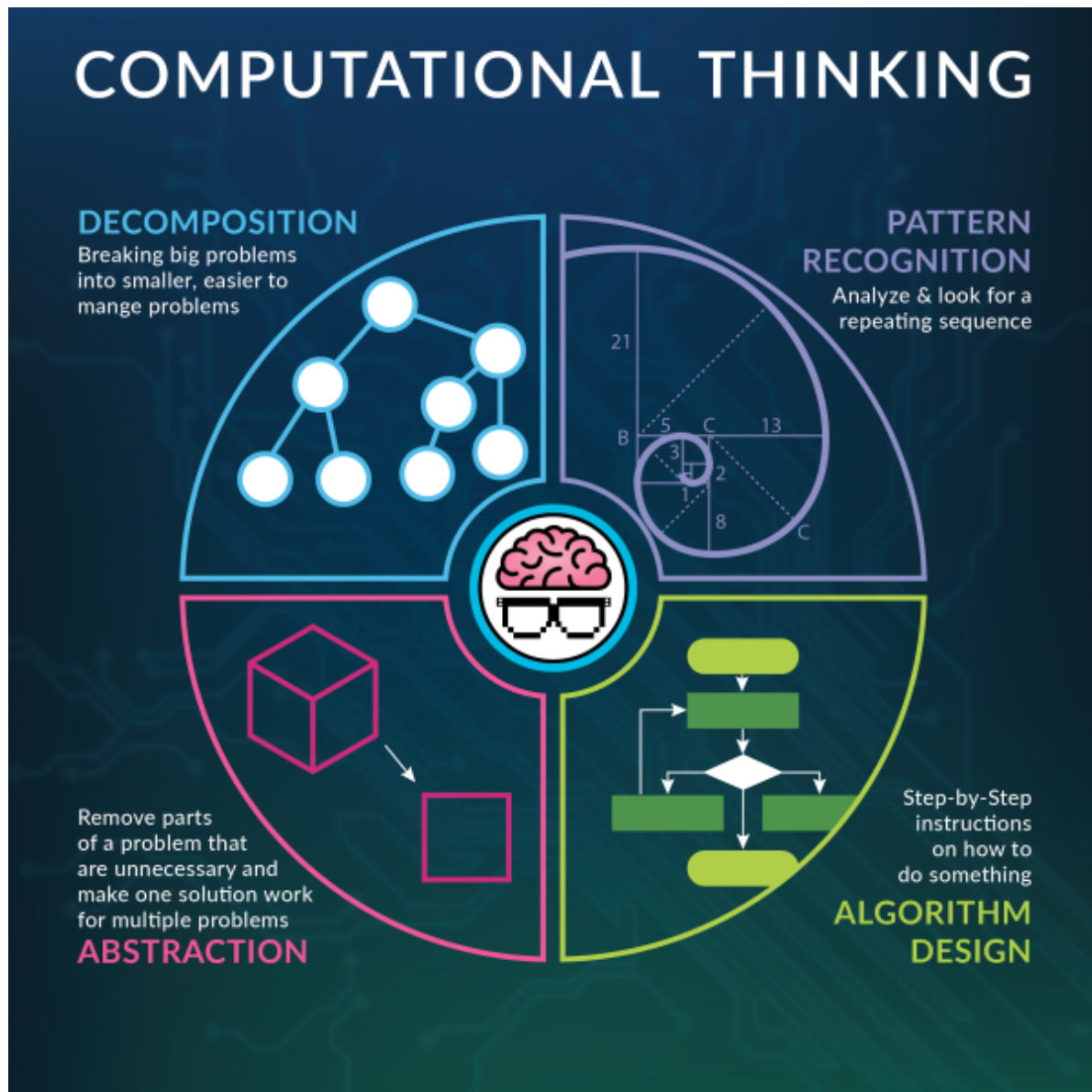


Image created by [ISTE & CSTA](#)

Decomposition

Basic Structure

By the end of the quarter, instead of simply saying “I analyzed some data,” we can articulate more of what is involved as we learn how to do decompose what we do while learn R:

- I read some data
- I prepared the data for analysis
- I generated statistics and visualizations
- I modeled outcomes
- I communicated my findings to others

And Still More

and learn a little about a complete/professional workflow:

- Ingest data (read CSV / JSON / SQL database)
- Clean & make data consistent (e.g. fix various date formats)
- Address missing values
- Recode categories (regroup/relabel values)
- Transform values (logarithms, standardization)
- Engineer new variables (great creative skills)
- Aggregate or Reshape/restructure data
- Model / analyze the cleaned data
- Validate/verify/reproduce results
- Communicate results (tables, figures, reports)
- Return to previous steps to refine one's understanding (iterative)

Each step is clear and repeatable across different data & problems

Pattern Recognition

Intuitively seeking structure, repetition, regularity in data:

- noticing the same data-cleaning steps apply to many different types of data files
- recognizing that bootstrapping, permutation tests, and cross-validation share a common resampling pattern (upper division)
- seeing that multiple models differ only by a formula or parameter

This is where the need for loops, reusable functions, and pipelines (making raw data analyzable) arise.

If I have time, the streetwise soldier who intuition saved his platoon during the worst of the Vietnam War (1965-1973).

Abstraction

Focus on what matters, minimize what does not:

- data is “rows = cases, columns = variables” rather than obsessing over file format
- thinking in terms of distributions rather than individual observations
- defining a function that works for “any dataset”

Abstraction is what helps you generalize from one analysis to the next. This helps maximize productivity and minimize time spent in repetitive tasks.

Algorithmic thinking, good design

Create a clear, step-by-step procedure that produces a (reproducible) result

- an iterative fitting routine
- simulation steps (“repeat 10,000 times...”)
- a reproducible analysis script that always gives the same output from the same input

This is the bridge between statistical reasoning and actual computation.

Why do this?

The American Statistical Association emphasizes computational thinking because modern statistics is no longer just formulas on paper:

- Simulation > closed-form solutions (formulas)
- Resampling (let the data tell us) > asymptotics (trust the math)
- Reproducible pipelines > one time calculations/analyses
- Data wrangling > textbook-clean datasets

statistical thinking + computational thinking = data science.

Workflow in Action & computational thinking

- Decomposition is like the flow of data and thought from problem to solution
- Pattern recognition is like seeing that flow across different projects
- Abstraction is like “cleaning” & “modeling” & “reporting” generally
- Algorithmic thinking is about having ordered, executable steps

Computational Thinking forces us to stop thinking in terms of isolated/one-off projects and start thinking in terms reproducible processes and the enterprise (whatever it may be) as a whole

Example: Estimate uncertainty in a statistic using resampling methods

Computational thinking

- Identify the statistic of interest (mean, median, slope)
- Define a resampling procedure
- Decide on the number of repetitions
- Decide how the results should be summarized
- Decide how uncertainty will be reported/communicated

Pure coding

- Write the loop / vectorized call
- Select random seeds
- Store the results
- Call a visualization function
- Debug syntax errors

Comparison

Computational thinking	Coding
Conceptual	Concrete
Language-independent	Language-specific
About structure	About syntax
Can be sketched	Must be run
Transfers across tools	Does not fully transfer

Recap: In Statistics 20

- Developing your computational thinking
- Applying computational thinking fundamentals using a language called R
- Statisticians apply procedures and are not limited to tools like R (e.g. Python, SQL, etc)
- Software changes but your thinking skills will endure
- We need to explain the steps and reasoning behind an analysis not just name the tools
- While it is acceptable to use AI to assist, learn more than copying and pasting
- Let's learn what to do and why we do it
- BUT FIRST... attendance!

Activity #2: Attendance

- Everyone in here has a camera.
- This is how you get credit for attendance, upload a minimum of one photo per week.
- You cannot receive credit if it's only you in the photo
- And I'll start rejecting photos if its always you and the same people.
- Meet your classmates, take a photograph. Here is an example, with their permission of course



Figure 3: The Mhen of Statistics

Takeaways

- Stats 20 is an introduction to R and here – a Computational Thinking approach
- Course specifics are in the syllabus, note the deliverables, please watch the video
- A little basic R - objects - **everything** in R is an object AND objects have names
- operators (e.g., +, -) are objects, vectors are objects, functions `is()`, `str()` and `summary()` are objects
- Choose your tools: specifically an IDE (RStudio, Positron, VS Code, etc.)

Thank You!